

Санкт-Петербургский государственный университет

Аверин Павел Владиславович

Выпускная квалификационная работа

Система для анализа мультиагентных алгоритмов определения позиций объектов

Уровень образования: бакалавриат

Направление *09.03.04 «Программная инженерия»*

Основная образовательная программа *СВ.5080.2022 «Программная инженерия»*

Научный руководитель:
д.ф.-м.н., проф., проф. каф. СП О. Н. Граничин

Рецензент:
старший инженер-программист, ООО «КНС ГРУПП» К.В. Кучеров

Санкт-Петербург
2026

Saint Petersburg State University

Pavel Averin

Bachelor's Thesis

System for analysis of multi-agent algorithms for objects tracking

Education level: bachelor

Speciality *09.03.04 "Software Engineering"*

Programme *CB.5080.2022 "Software Engineering"*

Scientific supervisor:
Sc.D, prof. O. N. Granichin

Reviewer:
Senior Software Engineer at "KNS Group" K. V. Kuchеров

Saint Petersburg
2026

Оглавление

Введение	4
1. Постановка задачи	6
2. Сбор требований	7
3. Обзор	9
4. Архитектура	11
4.1. Архитектура системы	11
4.2. Пользовательский сценарий	12
5. Особенности реализации	14
5.1. Используемый стек технологий	14
5.2. Симуляция	16
5.3. Пользовательские аккаунты и алгоритмы	18
5.4. Визуализация	18
5.5. Реализованные алгоритмы	20
Апробация системы	27
Заключение	29
Список литературы	30

Введение

В современном мире информационные технологии и вычислительные системы стремительно развиваются. Вычислительная мощность устройств увеличивается с каждым годом, а их размеры уменьшаются.

В рамках работы рассматривается задача определения позиций (трекинг) объектов по расстояниям распределенных сенсоров в условиях помех. На фоне технологического прогресса решение такой задачи существенно усложнилось, поскольку объекты наблюдения стали более сложными, быстрыми и непредсказуемыми.

Традиционным подходом к решению задачи является централизованное решение, при котором выделяется единый вычислительный узел, собирающий данные с сенсоров, выполняющий необходимые вычисления и формирующий управляющее воздействие. У такого подхода можно выделить три слабых места:

- время на передачу данных от сенсоров до вычислителя;
- обработка данных и вычисления;
- время на передачу управляющего воздействия от вычислителя обратно к сенсорам.

Современные технологии позволяют перейти от этого метода к более новому и эффективному мультиагентному подходу, который устраняет ключевые недостатки централизованной архитектуры, такие как единая точка отказа и ограниченная масштабируемость. В его рамках сенсоры сами выступают в роли вычислительных узлов: они могут самостоятельно определять траекторию объектов, используя как собственные измерения расстояний до целей, так и данные, полученные от соседних сенсоров. Такой подход повышает устойчивость системы, снижает задержки при обработке информации и позволяет более гибко адаптироваться к изменениям в окружающей среде.

К сожалению, на сегодняшний день не существует единой универсальной системы с набором уже реализованных мультиагентных под-

ходов, которая позволяла бы быстро и эффективно протестировать новый мультиагентный алгоритм трекинга и корректно сравнить его с существующими аналогами в одинаковых условиях. Это существенно замедляет проведение исследований, так как исследователям приходится тратить значительное время на самостоятельную реализацию других алгоритмов при отсутствии открытого исходного кода, а также на подготовку симуляций и настройку экспериментальной среды.

1. Постановка задачи

Целью дипломной работы является разработать систему для анализа мультиагентных алгоритмов, определяющих позиции движущихся объектов по расстояниям, измеряемым распределенными сенсорами, в условиях помех. Для достижения этой цели были сформулированы следующие задачи.

1. Провести обзор существующих систем для анализа мультиагентных алгоритмов.
2. Сформулировать требования к разрабатываемой системе.
3. Разработать архитектуру системы на основе требований.
4. Реализовать прототип системы и интегрировать в него ряд существующих мультиагентных алгоритмов.
5. Провести апробацию прототипа системы.

2. Сбор требований

В результате взаимодействия с членами научной группы, занимающейся разработкой рассматриваемых алгоритмов, были сформулированы требования к разрабатываемой системе. Эти требования отражают как особенности предметной области, так и ограничения существующих инструментов.

1. **Поддержка пользовательских аккаунтов и загрузки алгоритмов.** Система должна обеспечивать возможность создания пользовательских аккаунтов, а также предоставлять интерфейс для загрузки и интеграции пользовательских алгоритмов. Это необходимо для проведения сравнительных исследований различных подходов в единой среде.
2. **Гибкая настройка параметров симуляции.** Должна быть предусмотрена возможность задания параметров сценария, включая количество целей и сенсоров, продолжительность моделирования, характер движения целей, а также параметры возмущений и помех. Это позволяет исследовать устойчивость алгоритмов в различных условиях.
3. **Визуализация результатов моделирования.** Система должна предоставлять средства для наглядного отображения результатов, включая траектории движения целей и эволюцию средне-квадратической ошибки. Визуализация является важным инструментом анализа качества алгоритмов.
4. **Поддержка пакетного запуска экспериментов.** Необходима возможность автоматизированного запуска серии сценариев с последующей агрегацией результатов. Это существенно упрощает проведение масштабных экспериментальных исследований.
5. **Поддержка сравнения результатов при различных параметрах.** Система должна обеспечивать средства для сравнения

результатов работы алгоритмов при различных настройках симуляции, что позволяет выявлять зависимости качества работы алгоритмов от всех настраиваемых параметров.

Приведенный ниже анализ отражает необходимость в специализированной системе, ориентированной на проведение воспроизводимых и масштабируемых экспериментов с мультиагентными алгоритмами трекинга в условиях возмущений и помех.

3. Обзор

В настоящее время существует ряд программных средств и библиотек, предназначенных для моделирования мультиагентных систем и трекинга объектов. Однако большинство из них либо ориентированы на централизованные методы обработки, либо не учитывают специфику рассматриваемой задачи — анализ мультиагентных алгоритмов в условиях возмущений и помех, не обладающих стандартными статистическими предположениями.

MATLAB Sensor Fusion and Tracking Toolbox [3] предоставляет широкий набор инструментов для моделирования сенсоров, трекинга объектов и оценки качества алгоритмов. К его преимуществам относятся развитая экосистема, наличие готовых реализаций алгоритмов и удобные средства визуализации. Однако этот инструмент в первую очередь ориентирован на централизованные подходы к обработке данных. Кроме того, система является закрытой и требует дорогостоящей лицензии, а интеграция пользовательских мультиагентных алгоритмов затруднена.

Repast (Agent-Based Modeling Toolkit) [5] представляет собой универсальную платформу для моделирования мультиагентных систем. К её преимуществам можно отнести гибкость и возможность описания широкого класса агентных моделей. Тем не менее, эта платформа в основном применяется в социально-экономических и биологических задачах и не содержит встроенных средств для моделирования сенсоров и алгоритмов трекинга. Также стоит отметить, что система в настоящее время активно не развивается и ограниченно поддерживается.

SIMLAN и симуляторы на базе Gazebo [7] широко используются в задачах робототехники для моделирования поведения роботов и тестирования алгоритмов навигации. Их основным преимуществом является высокая степень реалистичности физической симуляции и развитые средства интеграции с робототехническими платформами. Однако такие системы ориентированы на детальное моделирование физической среды и являются избыточными и сложными в настройке для

задач абстрактного анализа алгоритмов трекинга. Кроме того, они не предоставляют удобных средств для массового сравнительного анализа алгоритмов.

Для наглядного сравнения рассмотренных решений приведём таблицу (табл. 1).

Таблица 1: Сравнение существующих решений

Характеристика	MATLAB Toolbox	Repast	Gazebo / SIMLAN
Простота интеграции своих алгоритмов	-	+	-
Поддержка мультиагентных алгоритмов	±	+	±
Достаточный набор инструментов симуляции и моделирования	+	-	±
Визуализация результатов	+	±	+
Пакетные запуски и их сравнение	±	-	-
Инструменты моделирования помех	-	-	-
Сравнение работы различных сценариев	-	-	-

Вывод. Проведённый анализ показал, что существующие решения либо ориентированы на централизованные методы обработки данных, либо не предоставляют необходимых инструментов для моделирования сенсорных систем и трекинга, либо избыточны для рассматриваемой задачи. Кроме того, ни одна из систем не поддерживает в полной мере проведение экспериментов в условиях нестандартных статистических предположений.

Таким образом, существует необходимость в разработке специализированной системы, позволяющей интегрировать пользовательские мультиагентные алгоритмы, гибко задавать параметры симуляции и проводить их сравнительный анализ в условиях возмущений и помех произвольной природы.

4. Архитектура

4.1. Архитектура системы

На основе требований, сформулированных в разделе сбора требований, была разработана модульная архитектура системы, обеспечивающая гибкость, расширяемость и возможность проведения воспроизводимых экспериментов.

Общая структура системы представлена на (рис. 1).

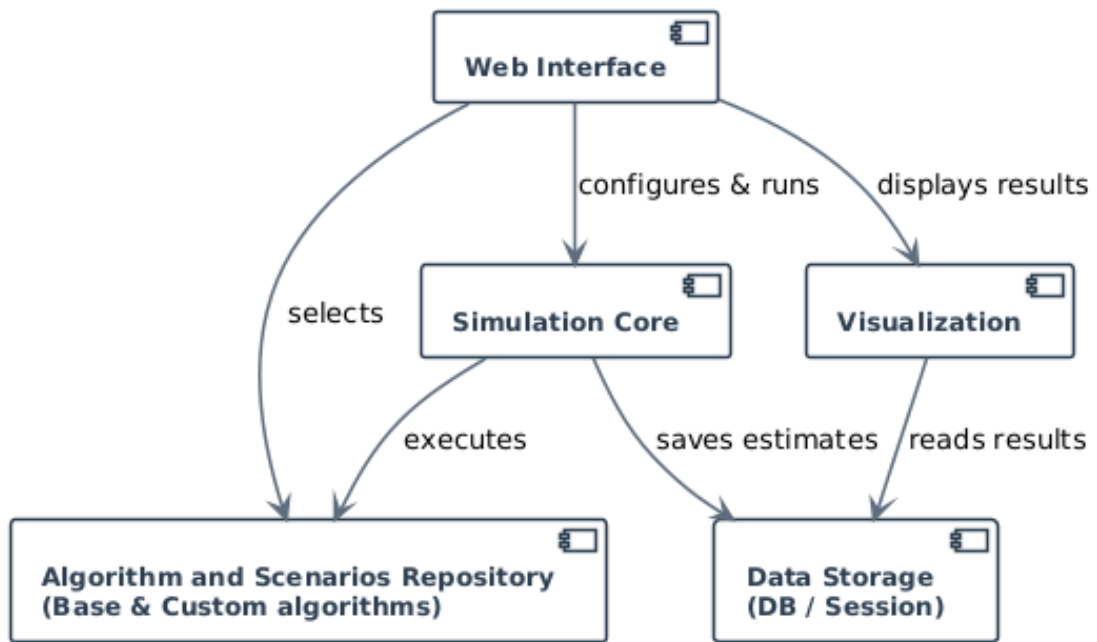


Рис. 1: Архитектура системы моделирования

Архитектура системы включает следующие основные компоненты:

- **Web Interface (веб-интерфейс).** Обеспечивает взаимодействие пользователя с системой. Через данный компонент осуществляется настройка параметров симуляции, выбор алгоритмов, запуск экспериментов и просмотр результатов.
- **Simulation Core (ядро моделирования).** Центральный компонент системы, отвечающий за выполнение симуляции, генерацию движения целей и обработку наблюдений с учётом шумов.

- **Algorithm Repository (репозиторий алгоритмов).** Содержит базовые и пользовательские алгоритмы и предоставляет унифицированный интерфейс для их выполнения.
- **Data Storage (хранилище данных).** Отвечает за сохранение результатов моделирования, параметров экспериментов и промежуточных вычислений.
- **Visualization (подсистема визуализации).** Обеспечивает отображение результатов в виде графиков траекторий и кривых ошибок.

Взаимодействие компонентов (рис. 1) организовано следующим образом: пользователь через веб-интерфейс задаёт параметры симуляции и выбирает алгоритмы, после чего управление передаётся в ядро моделирования. Ядро обращается к репозиторию алгоритмов, сохраняет результаты в хранилище и передаёт их в подсистему визуализации.

Таким образом, архитектура удовлетворяет требованиям: поддержки пользовательских алгоритмов, гибкой настройки, визуализации и пакетной обработки экспериментов.

4.2. Пользовательский сценарий

Типовой пользовательский сценарий работы с системой представлен на (рис. 2).

Сценарий работы пользователя включает следующие этапы:

1. **Открытие страницы настройки.** Пользователь переходит в веб-интерфейс и получает доступ к форме конфигурации.
2. **Задание параметров симуляции.** Пользователь задаёт длительность моделирования, количество целей и сенсоров, а также параметры шумов.
3. **Выбор алгоритмов.** Из репозитория выбираются алгоритмы для сравнения.

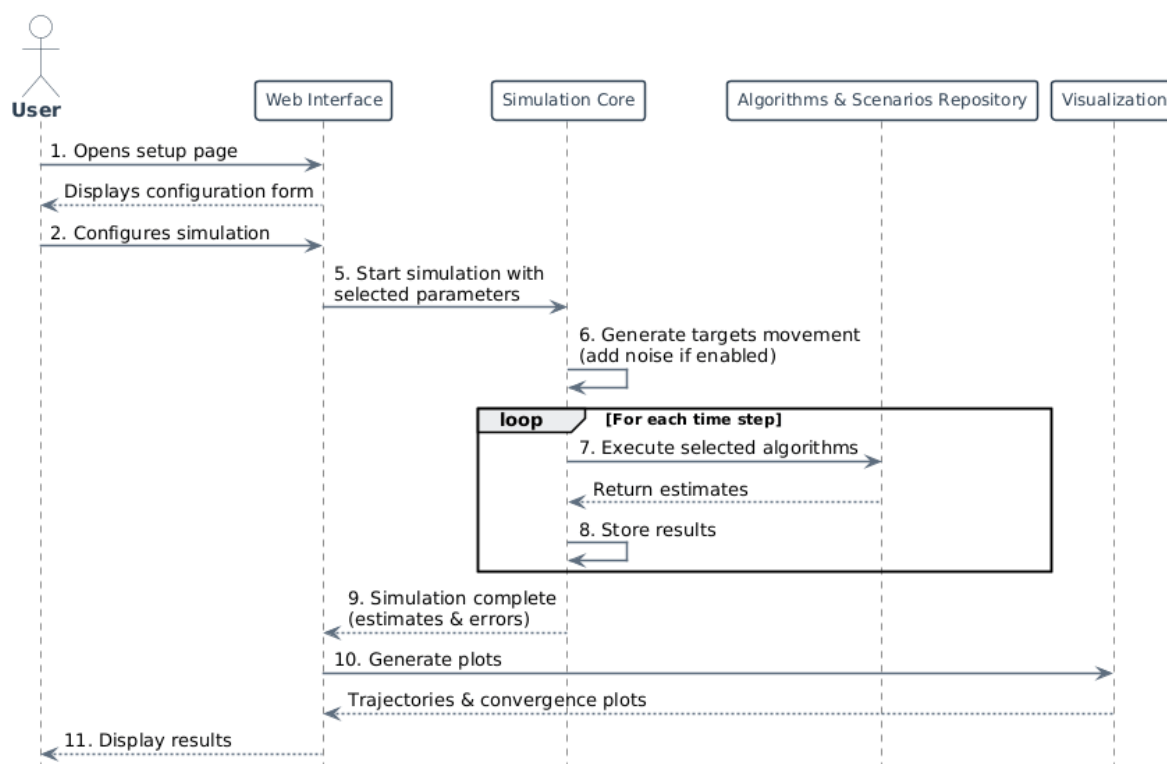


Рис. 2: Сценарий взаимодействия пользователя с системой

4. **Запуск симуляции.** Пользователь инициирует выполнение эксперимента.
5. **Выполнение моделирования.** Ядро моделирования выполняет расчёты, на каждом шаге времени вызывая выбранные алгоритмы.
6. **Получение результатов.** После завершения моделирования результаты передаются в систему визуализации.
7. **Анализ результатов.** Пользователь анализирует траектории, ошибки и сравнивает алгоритмы.

Дополнительно система поддерживает пакетный режим, позволяющий запускать серию экспериментов с последующим агрегированием результатов.

Таким образом, пользовательский сценарий (рис. 2) обеспечивает полный цикл работы с системой и соответствует сформулированным требованиям.

5. Особенности реализации

5.1. Используемый стек технологий

Язык программирования и среда выполнения. В качестве основного языка реализации выбран Python [21] версии 3.12 и выше. Выбор обусловлен следующими факторами:

- Использование языка разработчиками алгоритмов;
- Наличие мощных библиотек для научных вычислений (NumPy [14]) и визуализации (Matplotlib [13]);
- Развитая экосистема веб-фреймворков, в частности Django [17], используемого для реализации веб-интерфейса;
- Возможность динамической загрузки пользовательских модулей, что критически важно для поддержки расширяемости системы пользовательскими алгоритмами.

Веб-фреймворк и бэкенд. Для реализации серверной части и веб-интерфейса используется Django 5.2.7. Фреймворк обеспечивает:

- ORM для работы с базой данных и моделями пользователей;
- Встроенную систему аутентификации и авторизации;
- Маршрутизацию URL и систему шаблонов;
- Административную панель для управления пользователями и алгоритмами.

В качестве СУБД в проекте используется SQLite3 [12] (на этапе разработки) с возможностью переключения на PostgreSQL [11] через драйвер psycopg2 [18].

Библиотеки для научных вычислений и визуализации.

- NumPy 2.3.3 — для работы с многомерными массивами, выполнения матричных операций и линейно-алгебраических преобразований;

- `Matplotlib` 3.10.6 — для построения графиков траекторий движения целей и визуализации сходимости ошибок оценок;
- `SciPy` [15] (через зависимости `NumPy`) — для вычисления собственных чисел матриц и решения линейных систем.

Динамическая загрузка пользовательских алгоритмов. Ключевой особенностью системы является возможность загрузки пользовательских алгоритмов отслеживания. Для этого используется модуль `importlib.util`, который позволяет:

1. Динамически компилировать загруженный Python-файл;
2. Проверять наследование от базового класса `TrackingAlgorithm`;
3. Загружать модуль в память и вызывать его методы в процессе симуляции.

Загруженные алгоритмы сохраняются в модели `CustomAlgorithm` и ассоциируются с конкретным пользователем.

Фронтенд и визуализация. Для отображения веб-интерфейса используются:

- `Bootstrap` 5.3.0 [1] — для адаптивного оформления страниц;
- Встроенная система шаблонов `Django` с пользовательскими фильтрами;
- Генерация графиков в формате PNG с последующим кодированием в Base64 для встраивания в HTML-страницы без сохранения на диск.

Тестирование. Для обеспечения надежности системы используется фреймворк `pytest` 7.4.0 [16] с расширением `pytest-django` 4.5.2 [20]. Написаны следующие категории тестов:

- Модульные тесты алгоритмов;
- Интеграционные тесты взаимодействия симуляции и алгоритмов;

- Тесты с использованием мок-объектов для изоляции компонентов.

Инструменты разработки и CI/CD. Проект интегрирован с платформой `GitHub Actions` [2], что обеспечивает:

- Автоматическую проверку стиля кода (линтер);
- Запуск тестов при каждом пуше в репозиторий;

5.2. Симуляция

Процесс симуляции начинается с конфигурирования параметров через веб-интерфейс (рис. 3). Пользователь задаёт длительность симуляции, количество сенсоров, число линейных и случайных целей, а также количество запусков для статистической достоверности результатов. Для имитации реальных условий предусмотрена возможность добавления шума в измерения расстояний (равномерного или гауссовского) с настраиваемыми параметрами.

Multi-Agent Tracking Simulation Setup

[test](#)[Profile](#)[Logout](#)

Base algorithms

- ☒ Original SPSA
- ☒ Accelerated SPSA
- ☐ Distributed Kalman Filter

Noise Configuration

☐ Enable Distance Noise

Sensor Connectivity

☒ Generate random sensor topology

Density percentage

100

100% = fully connected, 0% = completely disconnected (sensors only communicate with themselves). Generated randomly.

☐ Use custom adjacency matrix

Simulation Configuration

Simulation Duration (seconds)

200

Number of Sensors

3

Target Configuration

Linear Targets

2

Random Walk Targets

2

Number of Simulation Runs

10

The simulation will run multiple times with different random seeds.

Run Simulation

Рис. 3: Интерфейс конфигурирования параметров симуляции

После запуска симуляции генерируются траектории движения целей (линейные или со случайным блужданием). Для каждого момента времени вычисляются истинные расстояния от каждого сенсора до каждой цели, после чего к ним применяется выбранная модель шума. Эти данные последовательно подаются на вход каждому из выбранных алгоритмов отслеживания, которые производят оценки позиций целей. Поддерживается как однократный запуск, так и пакетное выполнение нескольких сохранённых конфигураций для сравнительного анализа.

5.3. Пользовательские аккаунты и алгоритмы

Система поддерживает регистрацию пользователей с возможностью сохранения персональных конфигураций симуляции и загрузки собственных алгоритмов отслеживания. Пользовательские алгоритмы должны наследоваться от базового класса `TrackingAlgorithm` и реализовывать метод `run_n_iterations`. При загрузке файла производится проверка корректности интерфейса.

В проекте реализованы три встроенных алгоритма, описанные далее в работе:

- **Original SPSA + Consensus [4];**
- **Accelerated SPSA + Consensus [8];**
- **Distributed Kalman Filter [6].**

Пользовательские алгоритмы загружаются через веб-форму, после чего становятся доступными для выбора в интерфейсе настройки симуляции наравне со встроенными.

5.4. Визуализация

Результаты симуляции представляются в виде набора графиков, генерируемых с использованием библиотеки `matplotlib`. Для каждого запуска отображаются:

- Траектории истинного движения целей и оценки, полученные каждым алгоритмом (рис. 4).
- Графики сходимости ошибки, усреднённой по всем сенсорам для каждой цели.

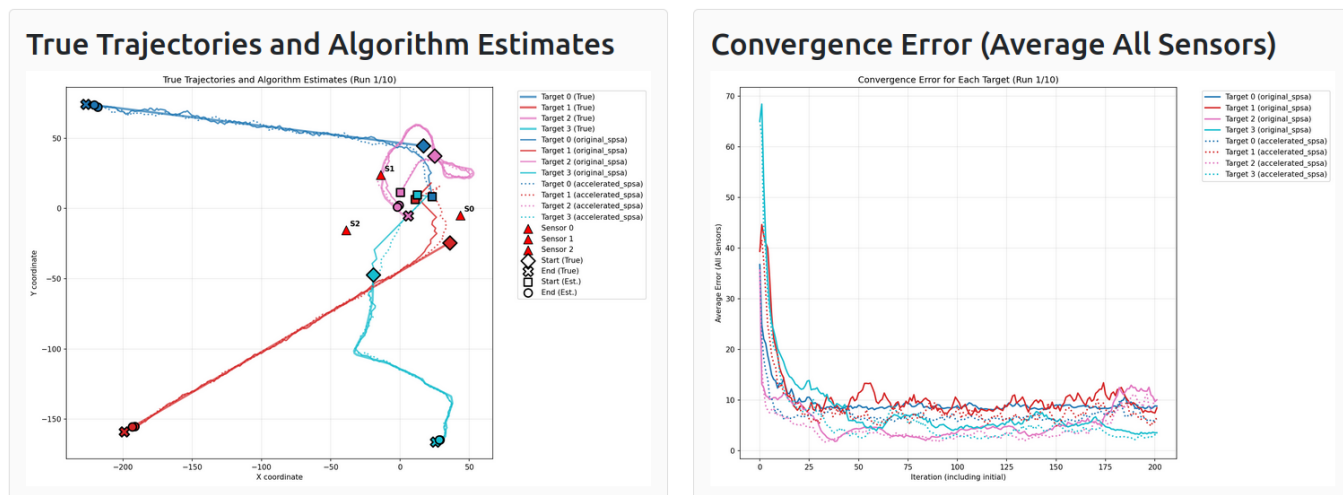


Рис. 4: Совмещённые траектории и ошибки для всех алгоритмов

Для детального анализа предусмотрен режим просмотра отдельных комбинаций сенсора и цели, где показывается траектория оценки конкретного сенсора за выбранной целью и соответствующая динамика ошибки (рис. 5).

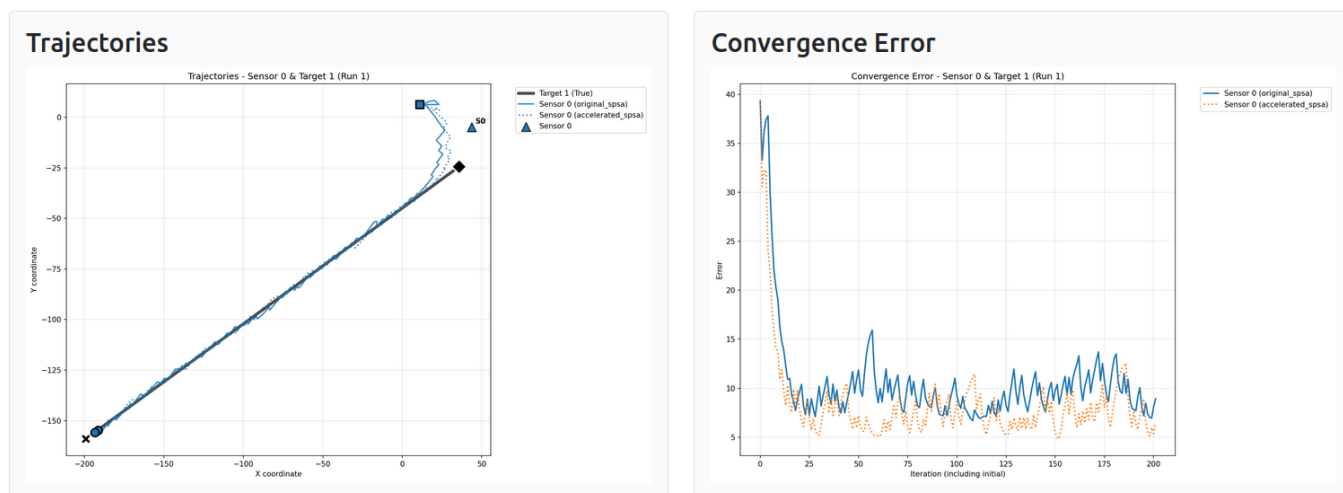


Рис. 5: Оценка конкретного сенсора за конкретной целью

При выполнении нескольких запусков одной конфигурации строится агрегированный график, показывающий среднюю ошибку и доверительный интервал (стандартное отклонение) по всем запускам (рис. 6). Это позволяет оценить устойчивость алгоритмов к случайным факторам.

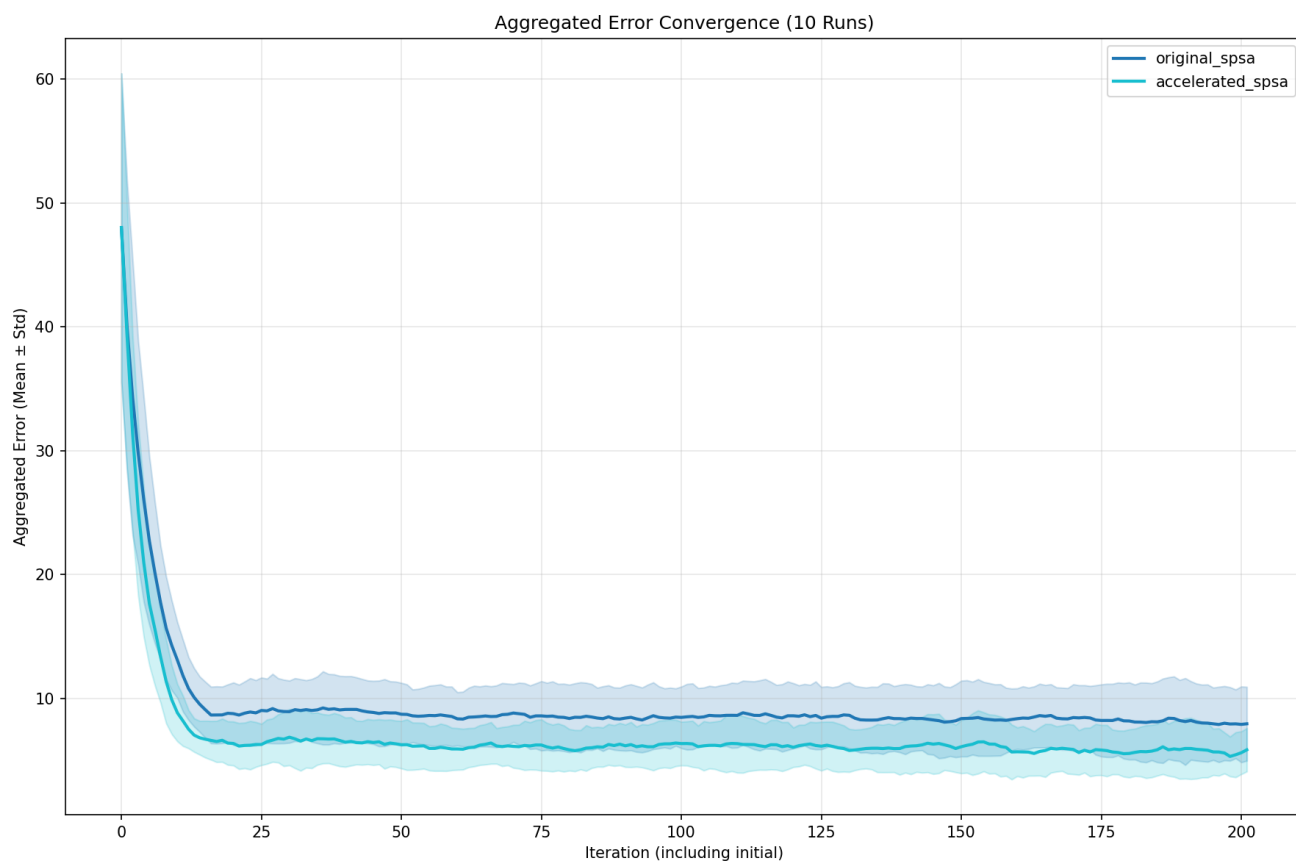


Рис. 6: Агрегированная ошибка по нескольким запускам

5.5. Реализованные алгоритмы

В рамках работы в систему интегрированы следующие мультиагентные алгоритмы определения позиций объектов.

Алгоритм на основе SPSA + консенсус

$$\left\{ \begin{array}{l} \mathbf{x}_{2k}^i = \hat{\boldsymbol{\theta}}_{2k-2}^i + \beta_k^+ \boldsymbol{\Delta}_k^i, \\ \mathbf{x}_{2k-1}^i = \hat{\boldsymbol{\theta}}_{2k-2}^i - \beta_k^- \boldsymbol{\Delta}_k^i \\ \hat{\boldsymbol{\theta}}_{2k-1}^i = \hat{\boldsymbol{\theta}}_{2k-2}^i, \\ \hat{\boldsymbol{\theta}}_{2k}^i = \hat{\boldsymbol{\theta}}_{2k-1}^i - \alpha \left(\frac{y_{2k}^i - y_{2k-1}^i}{\beta_k} \mathbf{K}_k^i(\boldsymbol{\Delta}_k^i) \right. \\ \quad \left. + \gamma \sum_{j \in \mathcal{N}_{2k-1}^i} b_{2k-1}^{i,j} (\tilde{\boldsymbol{\theta}}_{2k-1}^{i,j} - \hat{\boldsymbol{\theta}}_{2k-1}^i) \right) \end{array} \right.$$

Пояснение основных шагов.

1. **Генерация возмущений.** На каждой итерации k для каждого агента i генерируется случайный вектор возмущения $\boldsymbol{\Delta}_k^i \in \mathbb{R}^d$ с симметричным распределением. Этот вектор используется для аппроксимации градиента без явного вычисления производных.
2. **Формирование точек измерения.** Строятся две точки:

$$\mathbf{x}_{2k}^i = \hat{\boldsymbol{\theta}}_{2k-2}^i + \beta_k^+ \boldsymbol{\Delta}_k^i, \quad \mathbf{x}_{2k-1}^i = \hat{\boldsymbol{\theta}}_{2k-2}^i - \beta_k^- \boldsymbol{\Delta}_k^i.$$

Эти точки симметрично расположены относительно текущей оценки и используются для получения двух измерений функции.

3. **Получение наблюдений.** В точках $\mathbf{x}_{2k}^i$ и $\mathbf{x}_{2k-1}^i$ вычисляются (или наблюдаются) значения стохастической функции:

$$y_{2k}^i = f_{\xi_{2k}}^i(\mathbf{x}_{2k}^i), \quad y_{2k-1}^i = f_{\xi_{2k-1}}^i(\mathbf{x}_{2k-1}^i).$$

4. **Промежуточное обновление.** Оценка параметра на нечётном шаге не изменяется:

$$\hat{\boldsymbol{\theta}}_{2k-1}^i = \hat{\boldsymbol{\theta}}_{2k-2}^i.$$

Это делается для согласования с использованием пары измерений.

5. **Оценка псевдоградиента.** Градиент аппроксимируется по разности наблюдений:

$$\frac{y_{2k}^i - y_{2k-1}^i}{\beta_k} \mathbf{K}_k^i(\Delta_k^i),$$

где $\beta_k = \beta_k^+ + \beta_k^-$. Эта оценка требует всего двух измерений независимо от размерности задачи.

6. **Шаг согласования (консенсусная часть).** Второе слагаемое

$$\gamma \sum_{j \in \mathcal{N}_{2k-1}^i} b_{2k-1}^{i,j} (\tilde{\theta}_{2k-1}^{i,j} - \hat{\theta}_{2k-1}^i)$$

обеспечивает обмен информацией между агентами. Оно стремится согласовать оценки $\hat{\theta}^i$ с оценками соседей, полученными по шумным каналам.

7. **Обновление оценки.** Итоговое обновление имеет вид стохастического градиентного шага с добавлением консенсусного члена:

$$\hat{\theta}_{2k}^i = \hat{\theta}_{2k-1}^i - \alpha (\text{градиентная часть} + \text{консенсусная часть}).$$

Параметр α задаёт шаг обучения, а γ — степень взаимодействия между агентами.

Ускоренный алгоритм на основе SPSA + Консенсус

$$\left\{ \begin{array}{l} \bar{\mathbf{x}}_k = \frac{1}{\gamma_{k-1} + \alpha_k(\mu - \eta)} \left(\alpha_k \gamma_{k-1} \bar{\mathbf{z}}_{k-1} + \gamma_k \bar{\boldsymbol{\theta}}_{2k-1} \right), \\ \bar{\mathbf{x}}_{2k} = \bar{\mathbf{x}}_k + \beta_k^+ \bar{\Delta}_k, \quad \bar{\mathbf{x}}_{2k-1} = \bar{\mathbf{x}}_k - \beta_k^- \bar{\Delta}_k, \\ \bar{\mathbf{g}}_k = \left(\frac{\bar{\mathbf{y}}_{2k} - \bar{\mathbf{y}}_{2k-1}}{\beta_k} \otimes \mathbf{I}_d \right) \bar{\Delta}_k + \omega (\bar{\mathcal{L}}_{2k-1} \otimes E_n) \bar{\mathbf{x}}_k, \\ \bar{\boldsymbol{\theta}}_{2k} = \bar{\mathbf{x}}_k - h \bar{\mathbf{g}}_k, \\ \bar{\boldsymbol{\theta}}_{2k+1} = \bar{\boldsymbol{\theta}}_{2k}, \\ \bar{\mathbf{z}}_k = \frac{(1 - \alpha_k) \gamma_{k-1} \bar{\mathbf{z}}_{k-1} + \alpha_k (\mu - \eta) \bar{\mathbf{x}}_k - \alpha_k \bar{\mathbf{g}}_k}{\gamma_k} \end{array} \right.$$

Пояснение основных шагов.

1. **Формирование ускоренной опорной точки.** На каждой итерации строится точка $\bar{\mathbf{x}}_k$, которая является взвешенной комбинацией:

- предыдущего вспомогательного состояния $\bar{\mathbf{z}}_{k-1}$ (накопленная информация),
- текущей оценки $\bar{\boldsymbol{\theta}}_{2k-1}$ (актуальное состояние).

Такое смешивание реализует эффект ускорения (аналог метода Нестерова): учитывается как «инерция» прошлых шагов, так и новое направление поиска.

2. **Генерация возмущений.** Формируется случайный вектор $\bar{\Delta}_k$, компоненты которого имеют симметричное распределение. Он используется для аппроксимации градиента без явного вычисления производных (SPSA-подход).

3. **Формирование точек измерения.** Как и в классическом SPSA, строятся две симметричные точки:

$$\bar{\mathbf{x}}_{2k} = \bar{\mathbf{x}}_k + \beta_k^+ \bar{\Delta}_k, \quad \bar{\mathbf{x}}_{2k-1} = \bar{\mathbf{x}}_k - \beta_k^- \bar{\Delta}_k.$$

Они используются для получения двух наблюдений функции.

4. **Получение наблюдений.** В точках $\bar{\mathbf{x}}_{2k}$ и $\bar{\mathbf{x}}_{2k-1}$ вычисляются значения функции:

$$\bar{\mathbf{y}}_{2k}, \quad \bar{\mathbf{y}}_{2k-1},$$

которые могут содержать шум как измерений, так и коммуникаций между агентами.

5. **Оценка псевдоградиента.** Формируется оценка градиента:

$$\left(\frac{\bar{\mathbf{y}}_{2k} - \bar{\mathbf{y}}_{2k-1}}{\beta_k} \otimes \mathbf{I}_d \right) \bar{\Delta}_k,$$

которая дополняется консенсусным слагаемым:

$$\omega \left(\bar{\mathcal{L}}_{2k-1} \otimes E_n \right) \bar{\mathbf{x}}_k.$$

Первое слагаемое отвечает за локальную оптимизацию (SPSA), второе — за согласование оценок между агентами сети.

6. Обновление оценки. Новая оценка параметров вычисляется как

$$\bar{\boldsymbol{\theta}}_{2k} = \bar{\mathbf{x}}_k - h \bar{\mathbf{g}}_k.$$

Это шаг стохастического градиентного спуска, где направление задаётся оценённым градиентом.

7. Фиксация промежуточного состояния. На следующем шаге значение не изменяется:

$$\bar{\boldsymbol{\theta}}_{2k+1} = \bar{\boldsymbol{\theta}}_{2k}.$$

Это необходимо для согласования с двухточечной схемой SPSA.

8. Обновление переменной. Переменная $\bar{\mathbf{z}}_k$ обновляется по рекуррентной формуле:

$$\bar{\mathbf{z}}_k = \frac{(1 - \alpha_k) \gamma_{k-1} \bar{\mathbf{z}}_{k-1} + \alpha_k (\mu - \eta) \bar{\mathbf{x}}_k - \alpha_k \bar{\mathbf{g}}_k}{\gamma_k}.$$

Она аккумулирует информацию о прошлых шагах и играет роль «центра» вспомогательной квадратичной аппроксимации.

9. Формирование новой точки. На следующей итерации новая точка $\bar{\mathbf{x}}_{k+1}$ снова формируется как комбинация:

- устойчивой переменной $\bar{\mathbf{z}}_k$,
- текущей оценки $\bar{\boldsymbol{\theta}}_{2k}$.

Это обеспечивает ускоренную сходимость за счёт контролируемого накопления импульса.

Распределённый фильтр Калмана В основе алгоритма лежит двухэтапная процедура оценивания состояния динамической системы: локальное обновление по измерениям и распределённое согласование оценок соседних агентов с помощью итераций консенсуса.

Пусть $x(t) \in \mathbb{R}$ — скалярное состояние системы, $y_i(kT)$ — измерение i -го агента в момент kT . Алгоритм использует m обменов сообщениями между двумя последовательными измерениями. Оценка состояния на i -м узле обновляется следующим образом:

$$\begin{cases} \hat{x}_i(k|k) = (1 - \ell(k)) \hat{x}_i(k|k-1) + \ell(k) y_i(k), \\ \hat{x}_i(k + h\delta|k) = \sum_{j=1}^N Q_{ij}(k, h) \hat{x}_j(k + (h-1)\delta|k), \quad h = 1, \dots, m, \end{cases}$$

где:

- $\hat{x}_i(k|k)$ — оценка состояния на i -м агенте после обработки измерений в момент k ;
- $\hat{x}_i(k + h\delta|k)$ — промежуточная оценка на h -м шаге согласования;
- $\delta = 1/m$ — шаг между обменами сообщениями;
- $\ell(k)$ — скалярный калмановский коэффициент усиления (в стационарном случае $\ell(k) \equiv \ell$);
- $Q(k, h)$ — стохастическая матрица согласования, совместимая с графом коммуникаций (в стационарном случае $Q(k, h) \equiv Q$);
- N — число агентов.

Пояснение основных шагов.

- **Локальное обновление по измерениям.** Каждый агент i корректирует свою предсказанную оценку $\hat{x}_i(k|k-1)$ с использованием собственного измерения $y_i(k)$:

$$\hat{x}_i(k|k) = (1 - \ell) \hat{x}_i(k|k-1) + \ell y_i(k).$$

Это стандартный шаг калмановского обновления, не требующий коммуникаций.

- **Распределённое согласование (консенсус).** Затем в течение m итераций агенты обмениваются оценками с соседями и усредняют их:

$$\hat{x}_i(k + h\delta|k) = \sum_{j=1}^N Q_{ij} \hat{x}_j(k + (h-1)\delta|k), \quad h = 1, \dots, m.$$

Матрица Q выбирается стохастической ($Q\mathbf{1} = \mathbf{1}$) и неотрицательной, а её ненулевые элементы соответствуют рёбрам графа коммуникаций. Чем больше m , тем точнее оценки сходятся к глобальному среднему.

- **Предсказание на следующий шаг.** После завершения m консенсусных итераций оценка экстраполируется вперёд по времени (для скалярного процесса с белым шумом):

$$\hat{x}_i(k+1|k) = \hat{x}_i(k+m\delta|k),$$

после чего цикл повторяется.

Апробация системы

Апробация разработанной системы для анализа мультиагентных алгоритмов определения позиций объектов проводилась в рамках научно-исследовательской деятельности и включала представление результатов на научных мероприятиях, а также их публикацию в рецензируемых изданиях.

Основные результаты работы были представлены в виде доклада на XXVIII конференции молодых ученых «Навигация и управление движением» [9], организованной АО «Концерн ЦНИИ *Электроприбор*» [10]. Доклад был сделан в рамках секции «Прикладные задачи навигации и управления движением» и охватывал вопросы разработки архитектуры системы, реализации симуляционной среды, а также интеграции и сравнительного анализа мультиагентных алгоритмов. В ходе выступления были продемонстрированы возможности системы по проведению воспроизводимых вычислительных экспериментов, гибкой настройке параметров моделирования и визуализации результатов.

Работа также была представлена на конференции «СПИСОК-2026» [19] в секции «Комплексная автоматизация БПЛА: от планирования полётного задания до анализа данных».

Кроме того, по теме исследования был подготовлен реферат, который прошёл рецензирование и был принят к публикации в материалах указанной конференции. В публикации отражены ключевые положения работы, включая постановку задачи, описание архитектуры системы и результаты экспериментальных исследований.

Разработанная система была использована в качестве инструмента для проведения серии вычислительных экспериментов, направленных на сравнительный анализ алгоритма на основе SPSA с консенсусом и его ускоренной версии. Проведённые эксперименты включали пакетный запуск алгоритмов с помехами. Это позволило оценить сходимость алгоритмов.

Полученные результаты показали, что ускоренная версия алгоритма обладает более высокой скоростью сходимости при сохранении устойчи-

вости и точности оценивания, что подтверждает эффективность предложенных модификаций. Система обеспечила удобную платформу для проведения анализа за счёт поддержки пакетного запуска экспериментов и автоматизированного сравнения результатов.

Результаты проведённого исследования, полученные с использованием разработанной системы, были опубликованы в статье в журнале *IEEE Transactions on Automatic Control* [8], что подтверждает их научную значимость и новизну. Таким образом, разработанная система прошла апробацию как в рамках научных конференций, так и в процессе подготовки публикаций в высокорейтинговых научных изданиях.

Заключение

В ходе работы были получены следующие результаты.

- Проведён обзор систем MATLAB Sensor Fusion and Tracking Toolbox, Repast (Agent-Based Modeling Toolkit), SIMLAN.
- Разработаны требования, разбитые на следующие группы: генерация сценариев (сенсоры, объекты, помехи), поддержка алгоритмов (встроенных и пользовательских), сравнительный анализ (визуализация ошибок и траекторий).
- Спроектирована модульная архитектура системы, разделяющая симуляцию, выполнение алгоритмов и визуализацию.
- На основе предложенной архитектуры был реализован прототип системы в виде с использованием веб-фреймворка Django и SQLite в качестве базы данных, в который были интегрированы три мультиагентных алгоритма: оригинальный SPSA с консенсусом, его ускоренная версия и распределённый фильтр Калмана.

В качестве апробации результаты работы были представлены на XXVII конференции молодых ученых «Навигация и управление движением» и СПИСОК-2026. Графики, полученные с помощью системы, были включены в опубликованную статью в журнале IEEE Transactions on Automatic Control.

Исходный код разработанной системы: <https://github.com/Salvatore112/MultiagentTrackingAlgorithms>

Список литературы

- [1] Bootstrap. — URL: <https://getbootstrap.com/>. — (дата обращения: 4 мая 2026 г.).
- [2] Github Actions. — URL: <https://github.com/features/actions>. — (дата обращения: 4 мая 2026 г.).
- [3] MATLAB Sensor Fusion and Tracking Toolbox. — URL: <https://www.mathworks.com/products/sensor-fusion-and-tracking.html>. — (дата обращения: 4 мая 2026 г.).
- [4] Oleg Granichin, Senior Member, IEEE, and Natalia Amelina. Simultaneous Perturbation Stochastic Approximation for Tracking Under Unknown but Bounded Disturbances. — URL: https://www.researchgate.net/publication/277028715_Simultaneous_Perturbation_Stochastic_Approximation_for_Tracking_Under_Unknown_but_Bounded_Disturbances. — (дата обращения: 4 мая 2026 г.).
- [5] Repast. — URL: <https://repast.github.io/>. — (дата обращения: 4 мая 2026 г.).
- [6] Ruggero Carli, Alessandro Chiuso, Luca Schenato, Sandro Zampieri. Distributed Kalman filtering based on consensusstrategies. — URL: https://www.researchgate.net/publication/3236793_Distributed_Kalman_filtering_based_on_consensus_strategies. — (дата обращения: 4 мая 2026 г.).
- [7] SIMLAN. — URL: <https://github.com/infotiv-research/SIMLAN>. — (дата обращения: 4 мая 2026 г.).
- [8] Victoria Erofeeva, Oleg Granichin, Senior Member, IEEE, Anna Sergeenko. Accelerated Consensus-Based SPSA Algorithm for Multisensor Multitarget Tracking Problem. — URL: https://www.researchgate.net/publication/403227441_

[Accelerated_Consensus-Based_SPSA_Algorithm_for_Multisensor_Multitarget_Tracking_Problem](#). — (дата обращения: 4 мая 2026 г.).

- [9] XXVIII конференция молодых ученых «Навигация и управление движением» с международным участием. — URL: <https://elektropribor.spb.ru/young-scientists-conference/>. — (дата обращения: 4 мая 2026 г.).
- [10] АО «Концерн «ЦНИИ «Электроприбор». — URL: <https://elektropribor.spb.ru/>. — (дата обращения: 4 мая 2026 г.).
- [11] База данных PostgreSQL. — URL: <https://www.postgresql.org/>. — (дата обращения: 4 мая 2026 г.).
- [12] База данных SQLite3. — URL: <https://docs.python.org/3/library/sqlite3.html>. — (дата обращения: 4 мая 2026 г.).
- [13] Библиотека Matplotlib. — URL: <https://matplotlib.org/>. — (дата обращения: 4 мая 2026 г.).
- [14] Библиотека NumPy. — URL: <https://numpy.org/>. — (дата обращения: 4 мая 2026 г.).
- [15] Библиотека SciPy. — URL: <https://scipy.org/>. — (дата обращения: 4 мая 2026 г.).
- [16] Библиотека pytest. — URL: <https://docs.pytest.org/en/stable/>. — (дата обращения: 4 мая 2026 г.).
- [17] Веб-фреймворк Django. — URL: <https://www.djangoproject.com/>. — (дата обращения: 4 мая 2026 г.).
- [18] Драйвер psycopg2. — URL: <https://pypi.org/project/psycopg2/>. — (дата обращения: 4 мая 2026 г.).
- [19] Конференция СПИСОК-2026. — URL: <https://spisok.math.spbu.ru/2026/s4.asp>. — (дата обращения: 4 мая 2026 г.).

- [20] Плагин для pytest pytest-django. — URL: <https://pytest-django.readthedocs.io/en/latest/>. — (дата обращения: 4 мая 2026 г.).
- [21] Язык Python. — URL: <https://www.python.org/>. — (дата обращения: 4 мая 2026 г.).